

SURYAGRID AI

Full Project Dossier

Implemented History First, Then Complete Execution and Agentic Architecture

A team-owned record of what was implemented, when it entered the repository, how the current frontend/backend/agent system connects, and the complete controlled path forward.

Prepared by: SuryaGrid Project Team

Date: 9 July 2026

Repository baseline: 5bed3cd on main

Primary repository: <https://github.com/DBIT-Bangalore/SuryaGrid>

Team document. No individual authorship is asserted. Implemented claims are tied to repository history, current code, tests, model cards, or recorded verification. Planned claims are marked and gated by their required evidence.

Contents

I Everything Implemented to Date2

1 Verification Basis2

2 Current Implemented System3

3 Day-by-Day Implementation Record4

4 Implemented Workflow as of the Baseline6

5 Pull-Request and Branch Record6

6 Complete Main-Branch Commit Ledger7

7 Current Verification Result8

II Complete Execution and Agentic Architecture9

8 Outcome and Scope9

9 Full Frontend-Backend-Agent Architecture10

10 Frontend to Backend Connection Matrix10

11 Agentic Architecture and Agent Inventory11

12 Data, Models, Policy, and Source Plan13

13 Security, Reliability, and AI Guardrails14

14 Detailed Execution Roadmap14

15 Timeline and Milestones16

16 Standard Implementation Workflow16

17 Testing and Acceptance16

18 Cost and Capacity Planning17

19 Risks and Honest End State17

20 Immediate Team Actions17

21 External Technical References18

Part I

Everything Implemented to Date

1 Verification Basis

This dossier uses the following evidence hierarchy:

1. the current local checkout at `main` commit `5bed3cd`;
2. a fresh fetch from both configured GitHub remotes;
3. the GitHub repository commit list and pull-request metadata;
4. full local commit messages, file lists, and change statistics;
5. current code, routes, agents, data/model metadata, tests, and frontend build;
6. the existing evidence matrix and technical reports.

Repository history scope

The main branch contains **37 commits** from 20 June through 7 July 2026. The fetched object graph contains **39 unique commits**: the 37 main commits plus two side-branch commits not merged into main. The implementation history below records all 37 main commits and identifies the two non-main objects separately.

1.1 Current verified totals

Table 1: Current verified project baseline on 9 July 2026.

Measure	Verified value	Evidence
Main-branch commits	37	<code>git rev-list main --count</code> ; GitHub main at <code>5bed3cd</code>
Unique fetched commits	39	<code>git rev-list --all --count</code>
Merged development PRs	8	GitHub PRs 2 through 9
Backend route operations	61	FastAPI route table under <code>/api/v1</code>
Agent modules	17	<code>backend/app/agents/*.py</code> , excluding <code>__init__.py</code>
Frontend feature routes	12	page directories under <code>frontend/app</code>
Generated frontend pages	16	Next.js production build output, including root and framework pages
Backend tests	118 passed	<code>python -m pytest tests -q</code> , 19.30 seconds
Backend lint	all passed	<code>python -m ruff check app tests</code>
Frontend build	passed	Next.js 14.2.35 compile, type check, and static generation
Substation records	344	committed OSM-derived Bengaluru/Karnataka parquet
Current branch	<code>main</code>	both <code>origin/main</code> and <code>dbit/main</code> at <code>5bed3cd</code>

2 Current Implemented System

2.1 Backend

- FastAPI application with health, weather, sites, predictions, timelines, energy, settlement, RL, Karnataka, source, ML, agent, location, DSM, system, Kaggle, and substation-orchestration route groups.
- SQLAlchemy async persistence with SQLite default and PostgreSQL support; Alembic migrations, repositories, audit/training/settlement records, and health record counts.
- Redis integration for cache/rate state, provider status, and optional scheduled ingestion.
- Open-Meteo live/current/archive integration and pvlib solar position, transposition, cell-temperature, PVWatts, clear-sky, and Erbs decomposition paths.
- Deterministic energy, DSM, risk, fuzzy-risk, reward/settlement research, RL environment, provider/API management, and multi-agent orchestration.
- Real-data ML build/train/inference pipelines with provenance, quality, model registry, cards, training manifests, and reproducible Kaggle ingestion.
- Substation-driven context service and five dedicated endpoints that pass the selected substation through weather, solar, cloud, generation, and DSM.

2.2 Frontend

- Next.js 14 App Router with dashboard, predictions, timeline, energy, settlement, RL, Karnataka, locations, DSM, ML, data-sources, and system pages.
- A shared typed client in `frontend/lib/api.ts` with backend probe/offline handling and functions covering the 61-operation backend surface.
- Dark glass design system, metric/penalty cards, deviation/timeline charts, 3D SVG solar, energy-flow, and risk visuals, topbar/sidebar, and explicit backend-offline state.
- Substation workflow panel wired into Locations and DSM; model provenance and source status panels; Karnataka and RL views.

2.3 Data and models

- Open-Meteo Bengaluru historical weather, NASA POWER cross-check, OSM substations, India national/regional load references, and multiple Kaggle solar/PV/load datasets.
- Production-marked irradiance and cloud-risk models with cards; DSM-breach decision support.
- Kaggle Bengaluru irradiance and cloud models; India PV/load models retained as non-local, non-production evidence because of domain shift.
- RL artifacts and research environments exist, but the authoritative production card is blocked for insufficient real local load, economics, and PV outcomes.

2.4 Infrastructure and operations

- Dockerfiles and Docker Compose for frontend, backend, PostgreSQL, and Redis.
- GitHub Actions for backend tests, lint/format, frontend build, Pages, and an AWS CD path.
- Terraform for VPC, ECS/Fargate, RDS, ElastiCache, S3, CloudFront, security groups, and outputs.
- Single-instance Ubuntu deployment scripts, port changes, Nginx/subdomain configuration, and documentation. Managed AWS remains code-only until deployment is observed.

3 Day-by-Day Implementation Record

3.1 20 June 2026 - Phase 1 MVP and first product surface

Five commits: 6d08097, 957f146, d4b484f, 7e46fd8, 6e85caf.

- Created the initial 94-file Phase 1 repository: solar nowcasting, DSM penalty prediction, fuzzy-risk engine, FastAPI backend, Next.js frontend, schemas, tests, documentation, Docker/development assets, and the initial implementation plan.
- Rebuilt the dashboard with metric cards, penalty status, deviation bars, timeline charts, badges, page navigation, and a professional data-display hierarchy.
- Added the animated 3D SVG solar panel, risk gauge, energy-flow visualization, glassmorphism theme, responsive layout, and design tokens.
- Added static export, a labelled client-side demonstration engine, GitHub Pages workflow, and startup support.
- Removed temporary/scratch artifacts as the first cleanup step.

Day outcome: a complete demonstrable MVP and visual product shell existed, but its core data path was still primarily toy/synthetic and not yet the later real-data architecture.

3.2 24 June 2026 - UI consolidation

One commit: e092af5.

- Unified predictions and timeline pages into the dark glass system and removed broken light-theme class dependencies.
- Consolidated global card, badge, table, button, scrollbar, and animation styles.
- Added the Topbar, live/demo connection indicator, typography, landing-page feature highlights, and consistent metric/deviation/penalty components.

Day outcome: the frontend became one coherent application instead of visually separate pages.

3.3 26 June 2026 - Real physics, CI/CD, settlement, and RL foundation

Thirteen commits, including six feature/fix commits and their merge commits.

1. **Runtime correctness and root serving** (3627938, merged by 016b41c): fixed stale Redis health reporting, served the dashboard at root, removed the obsolete Pages workflow at that point, and updated run documentation.
2. **Continuous integration** (b872d09, merged by bb62673): added Python 3.12 backend tests and Node 20 frontend build/type-check jobs.
3. **Real nowcasting pipeline** (5054782, merged by f13bc3d): replaced the toy numeric path with Open-Meteo weather and pvlib physics; implemented solar position, POA, cell temperature, PVWatts DC/AC, clear-sky scheduling, excess-band DSM charging, deterministic risk, real weather/timeline APIs, and live frontend calls.
4. **CI/CD hardening** (6d7cf77, b283903, merged by 8bb5e0a): added Ruff lint/format, static-artifact validation, Pages deployment after CI, base-path detection, live URL verification, robust route-registration tests, and deployment docs.
5. **Irradiance decomposition** (a107eea, merged by 3950741): added automatic pvlib Erbs GHI-to-DNI/DHI fallback for providers without beam/diffuse data, while preserving measured DNI/DHI when available.
6. **Full main-plan foundation** (02ed09a, merged by 70c75cd): added reward/penalty/discount settlement research, energy balance, consumption profiles, provider rotation/quota tracking, RL digital twin and REINFORCE policy, policy weights, settlement/RL/energy APIs, and tests.

Day outcome: SuryaGrid moved from a visual/toy MVP to a real-weather, physics-based, continuously tested platform with the first complete deterministic decision loop and RL research foundation.

3.4 27 June 2026 - Persistence, Karnataka integration, real-time ingestion

Four commits: 5ce7bfe, 56beb46, f30e504, 6bb54f8.

- Added persistent async SQLAlchemy storage, portable GUIDs, the project ER model, repositories, startup initialization, migrations, and health record counts.
- Added Open-Meteo archive ingestion and a real-data RL training path over historical irradiance-derived production; persisted training runs.
- Added Energy Balance, Settlement Engine, and RL Lab frontend pages and client contracts.
- Added multi-year archive windows, current weather, current ingestion, and opt-in APScheduler.
- Added Karnataka solar-site presets, seed/region endpoints, a BESCOM connector framework, Karnataka DSM framework, related APIs, and a Karnataka page.
- Kept the BESCOM/SLDC connector explicitly simulated until real metered access exists.

Day outcome: the system acquired persistence, historical training, real-time ingestion, Karnataka-specific surfaces, and operational research workflows.

3.5 6 July 2026 - Phase 1.7 real-data and honest ML rebuild

One major commit: 95fb856; 113 files changed.

- Built a provenance-first data platform with source status, geography, data mode, domain shift, quality score, model cards, training manifests, and source registries.
- Built 26,304 hours of Bengaluru weather/solar history, a NASA POWER cross-check, 344 OSM substations, India load references, and solar/cloud/DSM/load training sets.
- Added solar, cloud-risk, DSM-breach, and load training/inference agents and endpoints.
- Marked local-unfit load/PV and RL honestly as non-production or blocked instead of promoting them from code existence alone.
- Added source, DSM, ML, agent, location, system, persistence, feature-engineering, and API management modules plus 16 Phase 1.7 tests.

Day outcome: the architecture became explicitly evidence-driven. “Real” was separated from “local”, “estimated”, “synthetic”, “unavailable”, and “production-ready”.

3.6 7 July 2026 - Kaggle ML, AWS assets, deployment, evidence reports, substations

Thirteen commits on main.

1. **AWS architecture** (507c6be): Terraform for VPC, ECS/Fargate, RDS, ElastiCache, S3/CloudFront, security groups; AWS deployment workflow and architecture, app-flow, and deployment docs.
2. **Single-instance deployment evolution** (1070a11, d4c7e62, 5c158c1, 7c0f22e, a17e3ef, 04f61d5): memory optimization for a small host, one-click Ubuntu deploy, backend port 8080, Nginx/subdomain attempts, direct-frontend fallback, and subdomain setup.
3. **Code/data support** (9b1c7eb, 9efbe28): formatted ML files and added a Kaggle download helper.
4. **Kaggle real-data pipeline** (16e6a4e): ingested PV, Bengaluru irradiance, cloud-risk, and India load datasets; built four prefixed models/cards; added Kaggle APIs, tests, evidence docs, and an implementation report.
5. **History integration** (df372d2): merged deployment/infra work with the Phase 1.7 Kaggle line.
6. **Evidence report** (360615f): added the final technical report, evidence matrix, formula/DSM source documents, and hosted-app verification.
7. **Substation-driven workflow** (5bed3cd): made a selected substation the central context; added context schema/service, seven-step orchestrator, five endpoints, 15 tests, frontend workflow panel, agent/calculation trace, honest blocking, and committed the small OSM/grid feature parquets.

Day outcome: the current **main** baseline was established: 61 APIs, 118 tests, real-data model provenance, substation-driven orchestration, deployment options, and detailed evidence documentation.

4 Implemented Workflow as of the Baseline

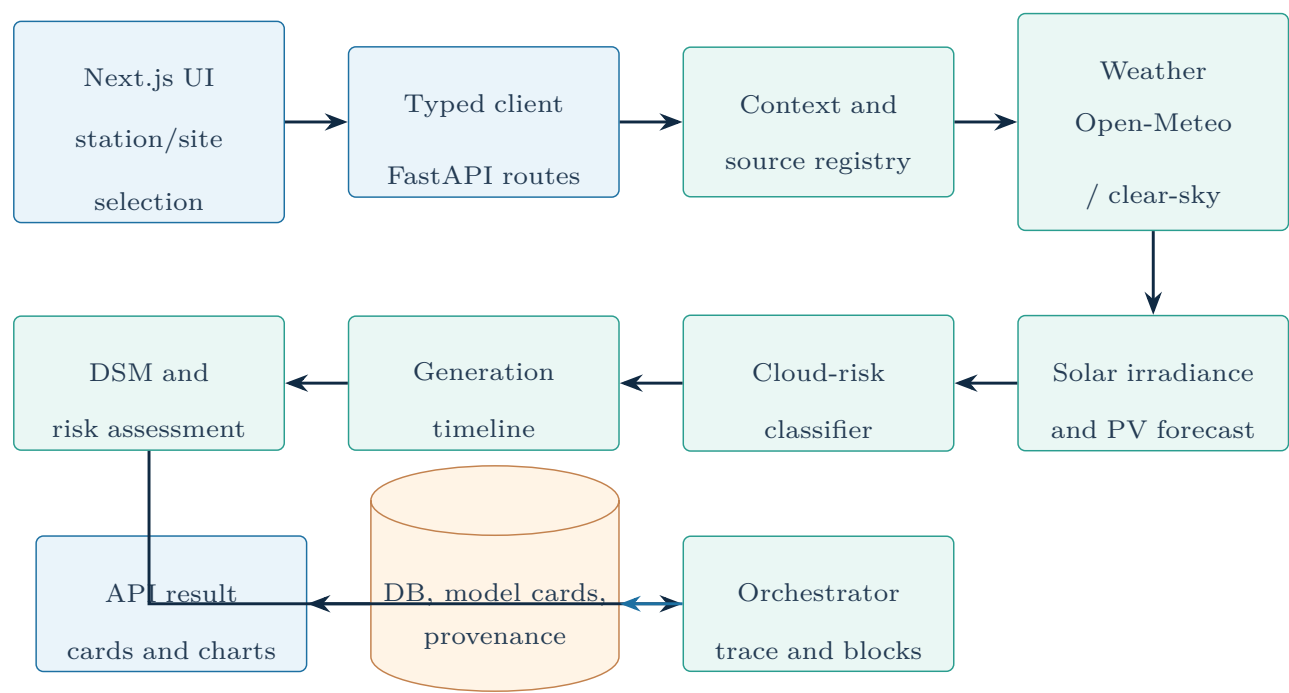


Figure 1: Current implemented substation/site decision-support loop.

Current honest limits:

- substation capacity is unavailable for all 344 OSM records;
- local measured PV and feeder/substation load are not connected;
- PV output is estimated from irradiance and configuration;
- official monetary DSM activation is pending source/version review;
- the production RL card remains blocked;
- authentication, tenant enforcement, alert delivery, read-only plant integration, HTTPS/operations evidence, and managed AWS deployment remain incomplete or unverified.

5 Pull-Request and Branch Record

Table 2: GitHub pull-request record relevant to main.

PR	Title / scope	Result	Main merge
2	Redis health and root path	merged	016b41c
3	CI backend tests and frontend build	merged	bb62673
4	Real Open-Meteo + pvlib nowcasting and DSM	merged	f13bc3d
5	Harden CI/CD and Pages	merged	8bb5e0a
6	Erbs fallback for GHI-only data	merged	3950741

PR	Title / scope	Result	Main merge
7	Settlement, energy balance, API agent, RL	merged	70c75cd
8	Persistent DB, historical RL, new UI	merged	56beb46
9	Karnataka/BESCOM, multi-year and current ingestion	merged	6bb54f8
10	CodeRabbit-generated unit-test branch	open; not in main	side commit 32eec8a

6 Complete Main-Branch Commit Ledger

Table 3: All 37 commits on main, in chronological order.

Commit	Date	Repository summary	Implementation meaning
6d08097	20 Jun	Phase 1 complete MVP	Initial backend, frontend, agents, DSM/fuzzy risk, tests, docs
957f146	20 Jun	Frontend UI overhaul	Dashboard, cards, deviation and timeline visuals
d4b484f	20 Jun	Temporary-file cleanup	Repository hygiene
7e46fd8	20 Jun	3D SVG/glass UI	Solar, risk, energy visuals and theme
6e85caf	20 Jun	GitHub Pages and mock engine	Static demonstration/deployment path
e092af5	24 Jun	Cohesive dark glass UI	Design-system and page unification
3627938	26 Jun	Redis health/root fix	Correct live health and root serving
016b41c	26 Jun	Merge PR 2	Integrated runtime fix
b872d09	26 Jun	CI workflow	Backend tests and frontend build
bb62673	26 Jun	Merge PR 3	Integrated CI
5054782	26 Jun	Real nowcasting + DSM	Open-Meteo, pvlib, real timeline, deterministic risk
f13bc3d	26 Jun	Merge PR 4	Integrated real nowcasting
6d7cf77	26 Jun	Harden CI/CD and Pages	Ruff, artifact/live checks, deployment docs
b283903	26 Jun	OpenAPI route test fix	FastAPI-version-stable route verification
8bb5e0a	26 Jun	Merge PR 5	Integrated hardened pipeline
a107eea	26 Jun	Erbs GHI fallback	Physically valid DNI/DHI derivation
3950741	26 Jun	Merge PR 6	Integrated Erbs fallback
02ed09a	26 Jun	Full platform foundation	Settlement, reward, energy, RL, API agent
70c75cd	26 Jun	Merge PR 7	Integrated full-platform modules
5ce7bfe	27 Jun	Persistence, historical RL, new UI	DB, archive training, energy/settlement/RL pages
56beb46	27 Jun	Merge PR 8	Integrated persistent data/UI
f30e504	27 Jun	Karnataka/BESCOM and current data	Multi-year/current ingestion and state workflow
6bb54f8	27 Jun	Merge PR 9	Integrated Karnataka expansion
95fb856	6 Jul	Phase 1.7 real-data ML	Provenance-first datasets, agents, cards, honest gates
507c6be	7 Jul	AWS IaC and documentation	ECS/RDS/Redis/S3/CloudFront intended stack
1070a11	7 Jul	2 GB host optimization	Small-instance Compose tuning
d4c7e62	7 Jul	Ubuntu deploy script	One-command server deployment workflow
5c158c1	7 Jul	Backend port 8080	Host/proxy port alignment
7c0f22e	7 Jul	Nginx proxy	Subdomain reverse-proxy attempt
a17e3ef	7 Jul	Direct frontend fallback	Resolved port-80 conflict path
04f61d5	7 Jul	Subdomain setup	Nginx site automation
9b1c7eb	7 Jul	ML formatting	Ruff code-format cleanup
9efbe28	7 Jul	Kaggle download helper	Dataset acquisition support
16e6a4e	7 Jul	Kaggle pipeline/models/report	Four datasets/models, APIs, cards, evidence

Commit	Date	Repository summary	Implementation meaning
df372d2	7 Jul	History merge	Combined infrastructure and Kaggle lines
360615f	7 Jul	Evidence-based final report	Hosted/repository verification and formula sources
5bed3cd	7 Jul	Substation-driven workflow	Context, orchestration, endpoints, UI panel, 118-test baseline

6.1 Fetched commits not in main

- 85c7753: side-branch CI commit associated with the Redis/root-path branch history.
- 32eec8a: CodeRabbit-generated tests on open PR 10; not merged into main.

7 Current Verification Result

On 9 July 2026, the current checkout was revalidated:

- `python -m pytest tests -q`: **118 passed**.
- `python -m ruff check app tests`: **all checks passed**.
- `npm run build`: **compiled, linted, type-checked, and generated 16 pages**.

This verifies the current repository baseline. It does not automatically verify production uptime, official regulatory interpretation, external datasets not committed to the repository, managed AWS deployment, or real plant telemetry.

Part II

Complete Execution and Agentic Architecture

8 Outcome and Scope

The target is a production-capable solar decision-support platform that connects weather and station data to forecast models, deterministic energy and DSM calculations, safe agentic reasoning, operator workflows, and eventually validated plant telemetry. The system must answer:

1. How much power can each DG or solar station generate over the selected horizon?

2. How does the forecast compare with schedule, request, demand, and measured actuals?

3. What energy gap, risk band, or regulatory DSM result follows?

4. What should an operator review or do next, with complete provenance and calculation trace?

5. Can a constrained RL policy improve advisory recommendations after real local outcomes exist?

I Dual truth model

Every response has a deterministic truth plane and an optional reasoning plane. The truth plane owns all source data, units, forecasts, formulas, model outputs, and money calculations. The reasoning plane may route tools, summarize, detect inconsistencies, and recommend review actions. It may never overwrite numeric results or invent missing values.

8.1 Four-level product model

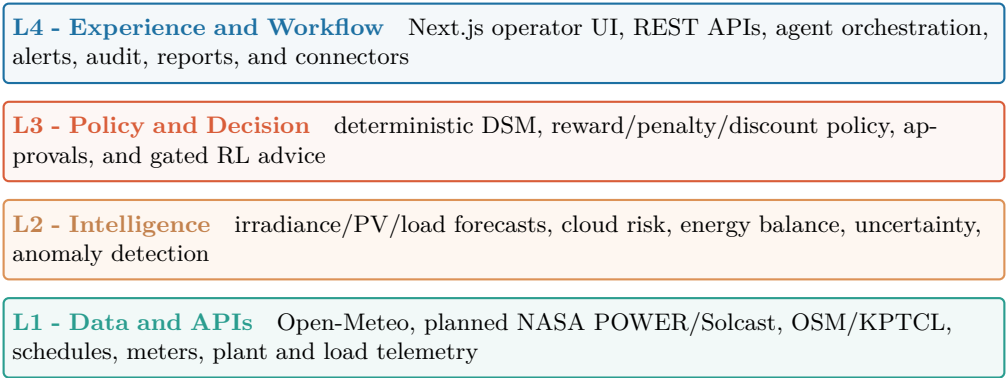


Figure 2: The complete four-level SuryaGrid product model.

9 Full Frontend-Backend-Agent Architecture

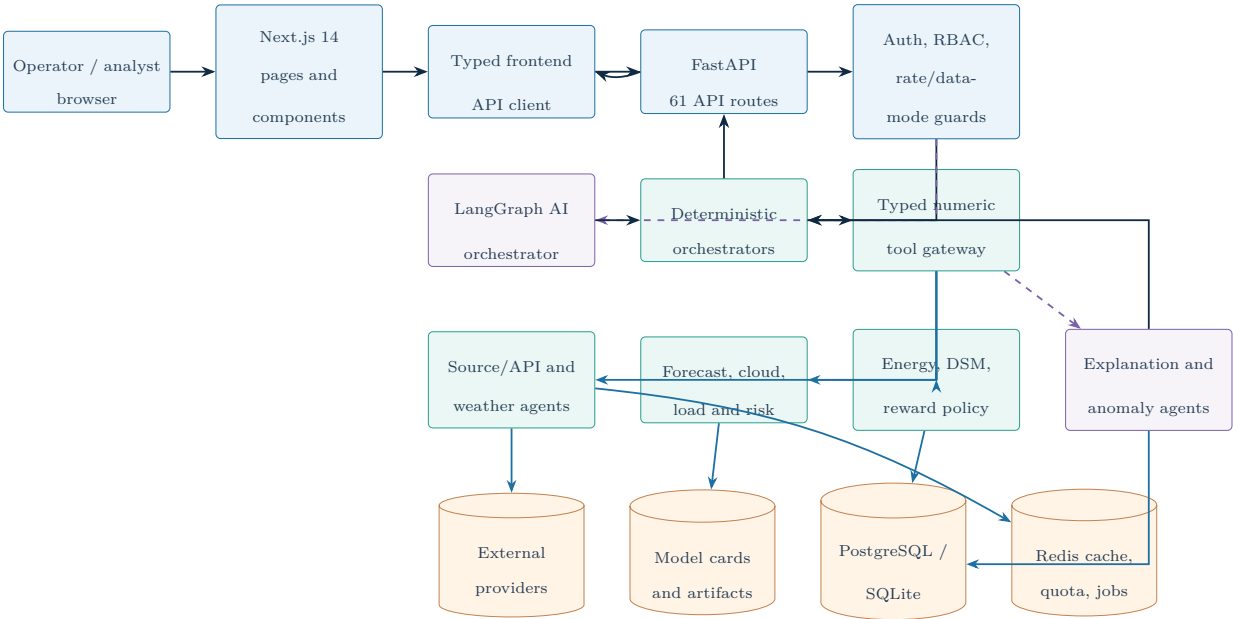


Figure 3: Target connection architecture. Solid navy paths are request/control; blue paths carry typed data; dashed violet paths are optional LLM reasoning.

9.1 Connection contract

1. The UI calls only the typed client in `frontend/lib/api.ts`; pages do not call providers or model files directly.
2. FastAPI validates request identity, role, site scope, data mode, units, and schema.
3. The deterministic orchestrator remains the production fallback and numerical authority.
4. The optional LangGraph route chooses from an allowlist of typed tools; tools call existing providers, models, energy services, and DSM engines.
5. Tool output is immutable and hashed into the trace before an LLM sees it.
6. The LLM returns schema-validated routing, explanation, anomaly, or escalation fields.
7. Persistence stores inputs, source IDs, model/formula/policy versions, outputs, agent trace, calculation trace, latency, cost, and approval status.
8. The API returns one stable response envelope to the UI. If LLM service is unavailable, numeric output still completes and the explanation is marked unavailable or deterministic.

10 Frontend to Backend Connection Matrix

Table 4: Frontend pages, backend contracts, and agent ownership.

Frontend surface	Primary client calls	Backend route groups	Agents/services
Dashboard	health, predict, orchestrate substation, generation timeline	/health, /predict, /orchestrate/substation, /generation/timeline	orchestrator, substation orchestrator, forecast, risk, explanation
Locations	catalog, context, nearest, coverage	/substations/*, /locations/*, /sites/*/data-coverage	location data, source registry, substation context
Predictions	predict site, forecast, model status	/predict/*, /forecast/*, /ml/model/status	forecast, feature engineering, model registry
Timeline	timeline, summary, generation timeline	/timeline/*, /summary/*, /generation/timeline	forecast, energy, DSM classifier

Frontend surface	Primary client calls	Backend route groups	Agents/services
Energy	energy and consumption profiles	<code>/energy/*</code> , <code>/consumption/profiles</code>	energy service, forecast, consumption service
DSM	profiles, advanced check, substation DSM, Karnataka DSM	<code>/dsm/*</code> , <code>/dsm/forecast</code> , <code>/dsm/karnataka/*</code>	DSM classifier, DSM engine, reward/policy
Settlement	settle day, settlement history, RL rates	<code>/settle*</code> , <code>/settlements/*</code> , <code>/rl/rates</code>	reward, persistence, energy/DSM services
RL Lab	train, rates, run history	<code>/rl/train</code> , <code>/rl/rates</code> , <code>/rl/runs</code>	RL environment/trainer, persistence; production use gated
ML	ingest, build, train, predict, status	<code>/ml/*</code> , <code>/kaggle/*</code>	Kaggle data, feature engineering, model registry
Data Sources	source catalog, provider status	<code>/sources/*</code> , <code>/data-sources/status</code> , <code>/providers/status</code>	source registry, API management, live weather
Karnataka	regions, seed, BESCO status, Karnataka DSM	<code>/karnataka/*</code> , <code>/bescom/status</code> , <code>/dsm/karnataka/*</code>	location, BESCO connector framework, DSM engine
System	health, system status, agent/data status	<code>/health</code> , <code>/system/status</code> , <code>/agents/*</code>	API management, source registry, all status providers

11 Agentic Architecture and Agent Inventory

11.1 Current deterministic and ML agents

Table 5: Existing agent modules and target role.

Module	Responsibility	Connects to	Status
<code>api_agent.py</code>	provider choice, failover, quota state	providers, Redis, orchestrator	PARTIAL
<code>api_management_agent.py</code>	provider health, retry, rate visibility	system/status routes	PARTIAL
<code>source_registry_agent.py</code>	source records and formula references	registry, provenance, UI	IMPLEMENTED
<code>live_weather_agent.py</code>	live weather and labelled fallback	Open-Meteo, Redis, DB	IMPLEMENTED
<code>kaggle_data_agent.py</code>	Kaggle ingestion and normalization	data pipeline and model builds	IMPLEMENTED
<code>location_data_agent.py</code>	locations, substations, coverage	OSM parquet, site services	IMPLEMENTED
<code>feature_engineering_agent.py</code>	quality checks and model features	dataset builder, ML models	IMPLEMENTED
<code>forecast_agent.py</code>	formula/ML/hybrid solar nowcast	weather, pvlib, model registry	IMPLEMENTED
<code>dsm_classifier_agent.py</code>	interval deviation classification	schedule, forecast/actual	IMPLEMENTED
<code>dsm_engine_agent.py</code>	rule-profile resolution and persistence	DSM repository and policy	PARTIAL
<code>risk_agent.py</code>	deterministic operational risk	forecast and DSM output	IMPLEMENTED
<code>fuzzy_risk_agent.py</code>	fuzzy risk inference	forecast/DSM features	IMPLEMENTED
<code>reward_agent.py</code>	reward/penalty/discount research engine	settlement and RL	GATED
<code>explanation_agent.py</code>	deterministic explanation text	final result envelope	IMPLEMENTED
<code>persistence_agent.py</code>	saves important records	DB repository and audit	IMPLEMENTED
<code>orchestrator_agent.py</code>	standard interval sequence	all numeric agents	IMPLEMENTED
<code>substation_orchestrator.py</code>	substation-driven seven-step flow	context, weather, ML, DSM	IMPLEMENTED

11.2 Planned LLM reasoning agents

Table 6: New agentic reasoning layer.

Agent	Allowed responsibility	Forbidden responsibility	Fallback
AI Orchestrator	select approved tools, route around missing inputs, request review, explain routing	changing numeric outputs, tariffs, schedules, or control set-points	deterministic orchestrator
AI Explanation	structured summary, evidence-linked narrative, operator recommendations	inventing causes, values, regulations, or confidence	deterministic explanation
Anomaly Detective	interpret deterministic anomaly flags, group incidents, suggest investigation	being the only detector or suppressing hard-rule alerts	rule-based detector

11.3 LangGraph execution design

Use a typed `StateGraph` with explicit `START/END`, conditional edges, durable checkpoints, bounded retries, and interrupt points for human approval. The design is:

1. **Analyze context:** deterministic validation creates allowed/missing/blocked fields.
2. **Choose tools:** AI orchestrator selects only tools whose prerequisites are present.
3. **Execute tools:** numeric tools run outside the model and return typed results.
4. **Validate:** schema, unit, bounds, provenance, and output hash checks run.
5. **Assess anomaly:** hard rules/statistics flag anomalies; LLM adds explanation only.
6. **Apply policy:** deterministic effective-dated DSM/reward profile runs.
7. **Interrupt if required:** unsupported money, low confidence, policy change, or control recommendation requires a human decision.
8. **Synthesize:** explanation agent returns strict JSON fields.
9. **Persist and publish:** state/checkpoint, audit, costs, and final envelope are saved.

11.4 Shared state and communication protocol

Table 7: Minimum shared state contract.

State group	Required fields	Writer
Identity	trace ID, user/role, tenant, station/site, request timestamp	API guard
Context	coordinates, timezone, units, capacity, schedule, policy ID, missing fields	context service
Sources	source IDs, retrieval time, license, raw hash, fallback and quality status	source/API agents
Forecast	GHI/PV/load intervals, confidence, model/formula versions	forecast tools
Decision	measured actual, schedule deviation, energy gap, DSM band/amount, blocked fields	logic and policy tools
Reasoning	route decision, explanation JSON, anomalies, recommended review	LLM agents
Governance	approval, interrupt reason, retry count, token/cost, latency, final status	orchestrator and persistence

Agent-to-agent communication uses state, not free-form hidden conversation. Every node appends an event to `agent_trace`; every numeric result appends source/formula/model details to `calculation_trace`. Errors are

typed as retryable, fallback-eligible, review-required, or blocked. No agent may silently convert a blocked result into a value.

12 Data, Models, Policy, and Source Plan

12.1 Provider and data priorities

Table 8: Data and policy integration priority.

Source/work	Current state	Planned implementation	Required gate
Open-Meteo	live weather and archive	retain as primary weather provider	provider SLO and cache
NASA POWER	historical cross-check only	runtime historical/NRT fallback provider, not a day-ahead forecast substitute	latency/freshness test
Solcast	not integrated	optional contracted forecast provider	contract, quota, cost
NPP/Grid India reports	public reports confirmed	feasibility parser for region/state generation and capacity; do not assume hourly Karnataka load until inspected	schema proof
KPTCL/Aikosh candidate	not verified in repo	inspect access, fields, licensing, current vs planned capacity, then match conservatively	source and match review
Local PV actuals	absent	KPCL/BESCOM/operator data agreement and ingestion contract	meter semantics and consent
KERC 2026 DSM	externally reported; not encoded from archived official source	effective-dated cumulative slab engine with source snapshot	official gazette + legal review
Load model	India-wide, non-production locally	train regional then Karnataka/site model	chronological holdout and domain review
RL policy	production blocked	offline constrained research then advisory shadow mode	real outcomes + approved economics

12.2 KERC 2026 handling

Published reports describe a solar tolerance band of 5%, cumulative charges of Rs. 0.25, Rs. 0.50, and Rs. 0.75 per unit across increasing deviation slabs, a scheduled-generation denominator, a deemed 1 MW schedule for zero-schedule injection, and an annual cap of 3 paise per unit times annual generation. These values are **requirements candidates**, not yet permission to emit settlement money.

Before `emits_rupee_values=true`:

1. archive the official Gazette/KERC document and checksum it;
2. record notification date, effective date, applicability, section/table references, amendment status, and legal reviewer;
3. implement cumulative slab arithmetic, annual cap, interval aggregation, zero-schedule handling, over/under-generation treatment, and version selection;
4. independently reproduce golden cases and compare against a manual worksheet;
5. add dual approval, rollback, and audit for rule activation.

13 Security, Reliability, and AI Guardrails

- OIDC/JWT authentication, RBAC, tenant and site isolation on every protected route.
- Secret manager for provider and OpenRouter keys; never expose keys to browser or logs.
- Request rate limits plus separate LLM token, spend, and concurrency budgets.
- Tool allowlists, JSON Schema output, prompt-injection filtering, and no arbitrary code tools.
- Deterministic numeric equivalence test: AI and non-AI routes return identical numeric fields.
- Model/provider pinning with explicit fallback list and recorded final model ID.
- LLM outage, timeout, malformed output, and budget exhaustion fall back deterministically.
- Human interrupts for money activation, rule changes, low-confidence recommendations, external messages, or any future control action.
- Immutable audit: prompt template version, tool inputs/outputs, response schema, token usage, latency, reviewer, and release version.
- HTTPS, backups, restore drills, provider outage tests, SLOs, data drift, model drift, and incident runbooks before pilot.

14 Detailed Execution Roadmap

14.1 Phase 0 - Foundation hardening (Week 1)

Work items

1. Enforce `APP_DATA_MODE` on every API and background job. Real mode forbids silent synthetic fallback; clear-sky physics is labelled separately.
2. Unify the source registry and model-card provenance into one `SourceRecord` contract.
3. Verify Docker Compose end-to-end, health checks, migrations, backend tests, frontend build, and evidence log.
4. Quarantine unsupported monetary defaults and add a test that forbids unapproved currency.

Deliverables: data-mode behavior matrix, unified registry, no-orphan-source tests, Docker verification log, and unchanged 118-test baseline or better.

14.2 Phase 1 - Provider and regulation integration (Weeks 2–3)

NASA POWER

- Add `app/providers/nasa_power.py` behind `WeatherProvider`.
- Map GHI, temperature, humidity, wind, and pressure; use explicit UTC/LST conversion.
- Cache responses, record freshness, and use it for historical fill/NRT fallback.
- Cross-validate against Open-Meteo; never call reanalysis a live forecast.

KERC rule parser

- Add effective-dated `kerc_dsm_2026.py` only after the official-source gate.
- Support cumulative slabs, annual cap, deemed schedule, applicability, and source citations.
- Add scenario tests, manual reproduction, frontend attribution, and rollback.

14.3 Phase 1.5 - Real agentic reasoning layer (Weeks 3–4)

- Add `langgraph`, `langchain-core`, and the selected OpenAI-compatible client.

- Create `app/agents/ai/state.py`, `tools.py`, `orchestrator.py`, `explanation.py`, `anomaly_detective.py`, `prompts`, and `policies`.
- Add `/api/v1/orchestrate/substation/ai` beside the deterministic route.
- Add strict structured output, durable checkpointing, retry limits, and approval interrupts.
- Add `AIInsightsPanel`, `anomaly/alert` UI, and reasoning trace display.
- Prove numeric equivalence and deterministic fallback under LLM failure.

14.4 Phase 2 - Data acquisition and enrichment (Weeks 5–6)

- Inspect NPP/Grid India report files before defining granularity; build fixture-driven parser.
- Obtain regional/Karnataka load from a source that actually exposes required intervals.
- Inspect KPTCL/Aikosh or official KPTCL/CEA capacity sources; distinguish planned augmentation from commissioned capacity.
- Match stations by normalized name, operator, voltage, and geography with manual review.
- Begin local PV data requests to KPCL, BESCO, and willing plant operators.
- Prepare a local-PV ingestion schema for timestamp, AC/DC power, irradiance, inverter state, quality flag, and meter provenance.

14.5 Phase 3 - Model rebuild and gated RL (Weeks 7–10)

- Train regional load with chronological splits and compare against the national baseline.
- Train site-level PV only when measured site truth and configuration are sufficient.
- Calibrate intervals, bias, drift, and seasonal/site performance; produce model cards.
- Build an RL environment from approved data. Estimated PV may support research but can never make the policy production-ready.
- Compare RL with no-action and deterministic baselines; require zero safety violations.
- Promote agent status only when its own evidence and acceptance gate passes.

14.6 Phase 4 - Product integration and hardening (Weeks 10–14)

- Complete OIDC/JWT, RBAC, tenant isolation, rate limits, audit, and spend controls.
- Make station selection a shared frontend context that drives every relevant page/card.
- Add alerts, acknowledgment, explanation provenance, and blocked-reason UX.
- Implement one read-only plant connector and a replay environment; no control commands.
- Verify HTTPS, actual deployment topology, backups, restore, CI/CD, metrics, traces, and SLOs.

14.7 Phase 5 - Controlled pilot (Weeks 15–22)

- Run a 1–3-site shadow pilot for at least 2–4 weeks.
- Compare forecast, schedule, measured actual, policy result, and operator decision.
- Review every alert, blocked result, model drift event, and LLM fallback.
- If RL passed its gate, show advice only; record human acceptance/rejection and outcome.
- Conduct incident, provider outage, backup/restore, policy rollback, and security drills.
- Hold a go/extend/stop production-readiness review with accepted residual risks.

15 Timeline and Milestones

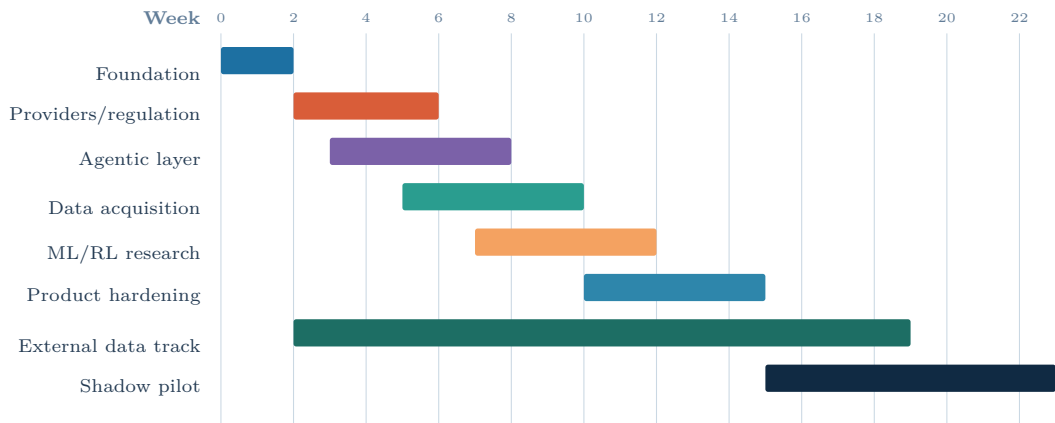


Figure 4: Integrated engineering and controlled-pilot schedule. The 10-week build track is inside the 22-week evidence and pilot track.

Table 9: Milestones and decision gates.

Week	Milestone	Success criterion	Decision
1	M0 Foundation	real-mode audit, registry design, Docker evidence, all tests/build pass	engineering gate
3	M1 Providers/rules	NASA provider validated; KERC source archived and golden cases reviewed	source/legal gate
4	M2 Agentic route	AI route returns schema-valid reasoning and identical numeric output; fallback proven	safety gate
6	M3 Data feasibility	inspected source files; coverage, interval, license, and quality report	data gate
10	M4 Model/RL research	signed model cards; RL beats baseline offline with zero safety breach	research gate
14	M5 Pilot-ready	auth/isolation, UI connection, read-only connector, SLO/restore drills	operational gate
22	M6 Pilot decision	shadow metrics, operator sign-off, incidents and residual risks reviewed	go/extend/stop

16 Standard Implementation Workflow

Every roadmap item follows the same evidence chain:

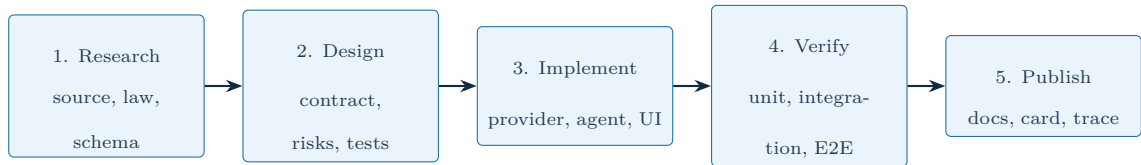


Figure 5: Required implementation workflow for every change.

17 Testing and Acceptance

- **Regression:** current 118 tests, Ruff, and frontend build remain green.
- **Provider contract:** fixtures, timezones, units, missing data, retries, cache, drift.
- **Policy:** official-source golden cases, boundaries, cumulative slabs, cap, rollback.
- **Agentic:** tool allowlist, prompt injection, structured output, numeric equivalence, deterministic fallback, checkpoint/resume, human interrupt, spend limit.

- **Data quality:** coverage, freshness, duplicates, gaps, license, geography, domain shift.
- **ML:** chronological holdout, baseline comparison, calibration, season/site slices, drift.
- **RL:** offline only, deterministic baseline, stress cases, constraints, no unsafe action.
- **Frontend/backend:** contract tests for all client functions and page workflows.
- **Security/operations:** auth, tenant isolation, secrets, rate limits, HTTPS, restore, provider outage, roll-back, load, SLO and incident drills.

18 Cost and Capacity Planning

Do not hardcode current model prices or model IDs in application logic. Select models from a capability policy that requires tool calling, structured output, latency, geography/privacy, availability, and budget. Record the actual model returned by the provider.

For planning:

$$\text{daily LLM cost} = \sum_a \left(\text{calls}_a \times \frac{\text{input tokens}_a \times p_{in,a} + \text{output tokens}_a \times p_{out,a}}{10^6} \right)$$

The supplied usage assumption of roughly 50 orchestrations, 50 explanations, and 24 anomaly reviews per day is a **pilot scenario**, not measured traffic. Add cache-hit rate, retries, provider fallback, and a 30–50% contingency. Set daily and monthly hard budgets; when exhausted, fall back to deterministic output.

19 Risks and Honest End State

Table 10: Principal execution risks.

Risk	Effect	Response
Official KERC text or interpretation changes	wrong monetary output	effective-dated rules, archived source, legal review, no-money default, rollback
NPP files lack required load granularity	load pipeline blocked	inspect first; obtain SLDC/site agreement; do not interpolate false hourly truth
Capacity dataset is planned work, not commissioned state	wrong station loading	source classification, commissioning evidence, confidence and manual review
Local PV partnership fails	no production PV/RL validation	retain estimate label; continue decision-support pilot without production RL
LLM changes numbers or invents evidence	unsafe decisions	typed tools, immutable outputs, strict JSON, equivalence tests, deterministic fallback
Prompt injection or secret leakage	data/security incident	allowlists, input separation, no secret in prompts, output validation, audit
Provider/model price or availability changes	cost/outage	model policy, pinned tested options, fallback list, budgets, cache
Frontend and backend contracts diverge	broken operator workflow	generated schemas, contract tests, one shared station context, CI build gate
Managed AWS is assumed deployed from Terraform	false readiness claim	verify actual topology; label code-only infrastructure until observed

After the plan, the system may still honestly report the following as pending: real-time KPTCL/SLDC telemetry without an agreement, on-site pyranometer validation, full local PV and load coverage, inter-state CERC settlement handling, autonomous control, and production RL. These are not documentation failures; they are evidence gates.

20 Immediate Team Actions

1. Approve this architecture and make it the source of truth for implementation.

2. Run the route-by-route APP_DATA_MODE and unsupported-currency audits.
3. Verify Docker Compose and record container evidence.
4. Implement NASA POWER as a historical/NRT fallback with explicit freshness semantics.
5. Archive and legally review the official KERC 2026 source before enabling any rupee output.
6. Create the typed LangGraph state/tool contracts and a deterministic-equivalence test first.
7. Inspect candidate NPP and KPTCL/Aikosh files before committing to schemas or schedules.
8. Send local PV data requests immediately because this is the longest external dependency.

21 External Technical References

- GitHub repository: <https://github.com/DBIT-Bangalore/SuryaGrid>
- LangGraph: <https://github.com/langchain-ai/langgraph>
- OpenRouter API: <https://openrouter.ai/docs/api/reference/overview>
- OpenRouter tool calling: <https://openrouter.ai/docs/guides/features/tool-calling>
- NASA POWER temporal API: <https://power.larc.nasa.gov/docs/services/api/temporal/>
- National Power Portal published reports: <https://www.npp.gov.in/publishedReports>
- Karnataka Electricity Regulatory Commission: <https://kerc.karnataka.gov.in/>